Redes neuronales convolutivas

M8: Fundamentos de deep learning

|**AE4**: Implementar un modelo predictivo utilizando redes neuronales convolutivas para el reconocimiento de imágenes en python.

alkemy

Introducción 🧑‍💻

Las redes neuronales convolutivas (CNN, por sus siglas en inglés) han transformado el campo del procesamiento de imágenes y visión por computadora. Inspiradas en el funcionamiento del sistema visual de los seres vivos, estas redes están especialmente diseñadas para reconocer patrones espaciales en imágenes mediante una combinación de operaciones matemáticas conocidas como convoluciones, funciones de activación no lineales, y reducción de dimensionalidad.

Este manual tiene como objetivo brindar una comprensión teórica y práctica de las CNN, desde sus fundamentos hasta su implementación con Python utilizando librerías como Keras. Se abordarán los elementos fundamentales de la arquitectura convolutiva, sus etapas y su aplicabilidad en problemas reales, así como estrategias de evaluación y optimización. Con estas herramientas, podrás crear modelos capaces de identificar objetos, clasificar imágenes y resolver problemas visuales con alto grado de precisión.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Describir en detalle qué es una red neuronal convolutiva y cómo se diferencia de una red neuronal densa.
- Explicar qué es una convolución y cómo actúa sobre una imagen.
- Implementar un modelo convolutivo simple en Python para reconocimiento de imágenes utilizando Keras.
- Optimizar un modelo CNN mediante evaluación de métricas, tuning de hiperparámetros y uso de técnicas como Dropout o aumento de datos.
- Interpretar las salidas del modelo y realizar predicciones con imágenes nuevas.

¿Qué es una red neuronal convolutiva?

Una red neuronal convolutiva (CNN) es una arquitectura de red especializada en el procesamiento de datos con una estructura de cuadrícula, como las imágenes. A diferencia de las redes neuronales densas (fully connected), que procesan los datos como vectores planos, las CNN mantienen la estructura espacial de los datos, lo que permite detectar patrones locales como bordes, texturas o formas.

Las CNN están compuestas por capas que realizan operaciones convolutivas, seguidas de funciones de activación, capas de reducción dimensional (pooling), y finalmente capas densas que realizan la clasificación. Esta combinación hace que sean muy eficientes para tareas de visión artificial, logrando resultados sobresalientes en clasificación de imágenes, detección de objetos, segmentación semántica y más.

Entre sus ventajas destacan la reducción del número de parámetros (gracias al uso de filtros compartidos), la detección jerárquica de características (de bajo a alto nivel), y su capacidad de generalizar bien en datos visuales complejos. Su arquitectura se inspira en el córtex visual humano, donde ciertas neuronas responden a patrones específicos en regiones específicas del campo visual.

¿Qué es una convolución?

La **convolución** es una operación matemática que combina dos funciones para producir una tercera. En el contexto de las CNN, se utiliza para aplicar filtros o kernels sobre una imagen, con el fin de extraer características específicas como bordes, esquinas o texturas.

Por ejemplo, al aplicar un filtro de 3x3 sobre una imagen, se multiplica cada valor del filtro con los valores correspondientes de un fragmento de la imagen, y luego se suman los productos. Este proceso se repite desplazando el filtro por toda la imagen (stride), generando un nuevo mapa de características.

Concepto	**Imagen**
Imagen de entrada	Matriz de pixeles

Filtro (kernel)	Matriz pequeña (ej. 3x3 o 5x5)
Resultado	Mapa de características (feature map)

La convolución permite extraer características locales sin necesidad de aplanar la imagen completa, lo cual conserva información espacial importante. A través de múltiples capas convolutivas, se pueden detectar patrones de bajo nivel (bordes) y patrones de alto nivel (rostros, objetos).

Campo de utilización de las redes neuronales convolutivas

Las CNN se utilizan principalmente en tareas donde la entrada tiene una estructura espacial, siendo las imágenes el ejemplo más común. Entre sus aplicaciones más destacadas están:

- **Clasificación de imágenes:** identificar si una imagen contiene un gato, perro, etc.
- **Detección de objetos:** localizar y clasificar objetos dentro de una imagen.
- **Reconocimiento facial:** autenticar usuarios mediante el análisis de rostros.
- **Segmentación semántica:** clasificar cada píxel de una imagen según su clase.
- **Medicina:** detección de tumores en imágenes de resonancia o rayos X.
- **Automoción:** visión computacional para conducción autónoma.

Su capacidad para detectar patrones visuales ha convertido a las CNN en la arquitectura estándar en visión por computadora, reemplazando muchas técnicas tradicionales basadas en ingeniería manual de características.

Etapas en las RNC: operación de convolución y capa ReLU

Una CNN se compone de varias etapas clave que permiten transformar progresivamente la imagen de entrada hasta generar una predicción:

1. **Operación de convolución:** como se explicó, aplica filtros para extraer características locales.

2. **Capa ReLU (Rectified Linear Unit):** introduce no linealidad, eliminando valores negativos y acelerando el entrenamiento.

La fórmula de ReLU es:
f(x) = max(0, x)

Esta etapa permite que la red aprenda relaciones no lineales entre los datos. En conjunto, las capas de convolución y ReLU generan mapas de activación que resaltan las regiones más relevantes de la imagen.

Múltiples combinaciones de convolución + ReLU permiten detectar características cada vez más complejas, lo cual es esencial para tareas avanzadas como reconocimiento de escenas o clasificación por contexto.

Capa de pooling

El **pooling** es una operación que reduce la dimensión espacial de los mapas de activación, manteniendo las características más importantes. El tipo más común es el **Max Pooling**, que toma el valor máximo dentro de una ventana deslizante.

Beneficios del pooling:

- Reduce la cantidad de parámetros.
- Disminuye el tiempo de entrenamiento.
- Ayuda a evitar el sobreajuste.
- Proporciona invariancia a pequeñas transformaciones espaciales.

Por ejemplo, un Max Pooling 2x2 reduce una imagen de 32x32 a 16x16, manteniendo lo más destacado en cada región.

Flattening y Full Connection

Una vez procesadas las imágenes por capas convolutivas y de pooling, es necesario convertir los mapas de activación en un vector unidimensional. Esto se conoce como **flattening**.

Este vector es luego pasado a una o más capas densas (**fully connected layers**), donde cada neurona está conectada a todas las neuronas de la capa anterior. Estas capas se encargan de realizar la clasificación o regresión final.

Etapa	Función principal
Flatten	Aplanar los mapas de activación
Full Connection	Interpretar características y generar salida

Las capas densas son similares a las usadas en redes neuronales tradicionales, y suelen ubicarse al final de las CNN como paso final de decisión.

Implementación de redes neuronales convolutivas en Python

Utilizando **Keras**, podemos construir una CNN de forma sencilla. A continuación, un ejemplo para clasificar imágenes de dígitos (dataset MNIST):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

modelo = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')  # 10 clases
])

modelo.compile(optimizer='adam',loss='categorical_crossentropy',
metrics=['accuracy'])
```

Este modelo puede entrenarse con imágenes en blanco y negro de tamaño 28x28. La red extrae características espaciales, las convierte en un vector y luego predice la clase.

Evaluación y optimización de la red neuronal convolutiva

Una vez entrenada, la red debe ser evaluada con datos no vistos. Se pueden utilizar métricas como:

- **Accuracy:** porcentaje de predicciones correctas.
- **Confusion matrix:** análisis detallado por clase.
- **Precision / Recall / F1-score**

Para optimizar el rendimiento, se pueden aplicar técnicas como:

- **Dropout:** evita el sobreajuste desactivando neuronas aleatoriamente.
- **Aumento de datos (data augmentation):** genera variantes artificiales de las imágenes.
- **Early Stopping:** detiene el entrenamiento si la red deja de mejorar.
- **Ajuste de hiperparámetros:** como learning rate, tamaño de lotes, etc.

Estas técnicas mejoran la capacidad de generalización del modelo, haciéndolo útil para trabajar con datos reales y variados.

Cierre ✍️

Las redes neuronales convolutivas son la herramienta por excelencia para abordar problemas de reconocimiento y clasificación de imágenes. Su arquitectura permite extraer patrones espaciales y jerárquicos de forma eficiente, logrando resultados superiores a métodos tradicionales. A lo largo de este manual, comprendiste desde los fundamentos de las operaciones de convolución, activación y reducción de dimensionalidad, hasta su implementación en Python mediante Keras.

El conocimiento adquirido te permitirá no solo construir modelos funcionales para tareas de visión por computadora, sino también evaluar su rendimiento y mejorarlos con técnicas de regularización y optimización. Las CNN son la base de tecnologías como el reconocimiento facial, la conducción autónoma y el diagnóstico por imagen, por lo que dominarlas es una habilidad clave para cualquier profesional del aprendizaje automático.

A medida que avances, podrás experimentar con arquitecturas más complejas como VGG, ResNet o MobileNet, e incluso trabajar con modelos preentrenados para tareas específicas. Lo aprendido aquí es la base sobre la cual podrás escalar a proyectos más ambiciosos y reales.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- TensorFlow. (2024). *Convolutional neural network (CNN)*. TensorFlow. Recuperado de https://www.tensorflow.org/tutorials/images/cnn
- Keras. (2024). *Introduction to CNNs*. Keras Documentation. Recuperado de https://keras.io/examples/vision/mnist_convnet/
- IBM. (2023). *What is a convolutional neural network?* IBM Cloud. Recuperado de https://www.ibm.com/cloud/learn/convolutional-neural-networks

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy